

RELATÓRIO DO PROJETO JACKUT – MILESTONE 01

Disciplina: Programação 2

Projeto: Jackut – Milestone 01

Linguagem: Java

Aluno: Artur Ferreira Marques da Silva

Professor: Mario Hozano Lucas de Souza

Período: 2026.1

1. Introdução

Este relatório apresenta a implementação do Milestone 01 do projeto Jackut, desenvolvido na disciplina de Programação 2, utilizando a linguagem Java. O objetivo do projeto foi construir, de forma incremental, uma rede de relacionamentos inspirada no Orkut, respeitando as histórias de usuário definidas para o primeiro marco da atividade.

No Milestone 01, o foco esteve na construção da base inicial do sistema, incluindo criação de usuários, abertura de sessão, consulta e edição de atributos de perfil, adição de amigos por meio de convites, efetivação de amizades e envio e leitura de recados entre usuários cadastrados.

O sistema foi implementado de forma compatível com os testes de aceitação disponibilizados para o projeto, contemplando as funcionalidades cobradas nas User Stories 1 a 4. Além disso, durante o desenvolvimento, buscou-se aplicar os principais conceitos estudados em sala de aula, especialmente os fundamentos de programação orientada a objetos, como encapsulamento, herança, abstração, composição, uso de listas, tratamento de exceções, uso de interfaces e organização em pacotes.

Este documento descreve a arquitetura geral do sistema, seus principais componentes, a interação entre eles, os conceitos de orientação a objetos aplicados, o padrão de projeto utilizado, a estratégia de persistência, os testes executados e as decisões de implementação adotadas para tornar o projeto compatível com diferentes ambientes de execução.

2. Descrição geral do design arquitetural do sistema

O projeto foi organizado de forma modular, separando as responsabilidades em diferentes pacotes e classes. A arquitetura escolhida teve como objetivo principal facilitar a compreensão do sistema, evitar concentração excessiva de responsabilidades em uma única classe de entrada e tornar o código mais legível, organizado e manutenível.

A estrutura principal do sistema foi dividida em:

- **modelos:** contém as entidades centrais do domínio do problema;
- **servicos:** contém classes responsáveis pela lógica de negócio e pela persistência;
- **contratos:** contém interfaces utilizadas pelo sistema;
- **excecoes:** contém a exceção personalizada utilizada no projeto;
- **Facade:** classe de acesso principal utilizada pelos testes de aceitação;
- **Main:** classe responsável por executar os testes via EasyAccept.

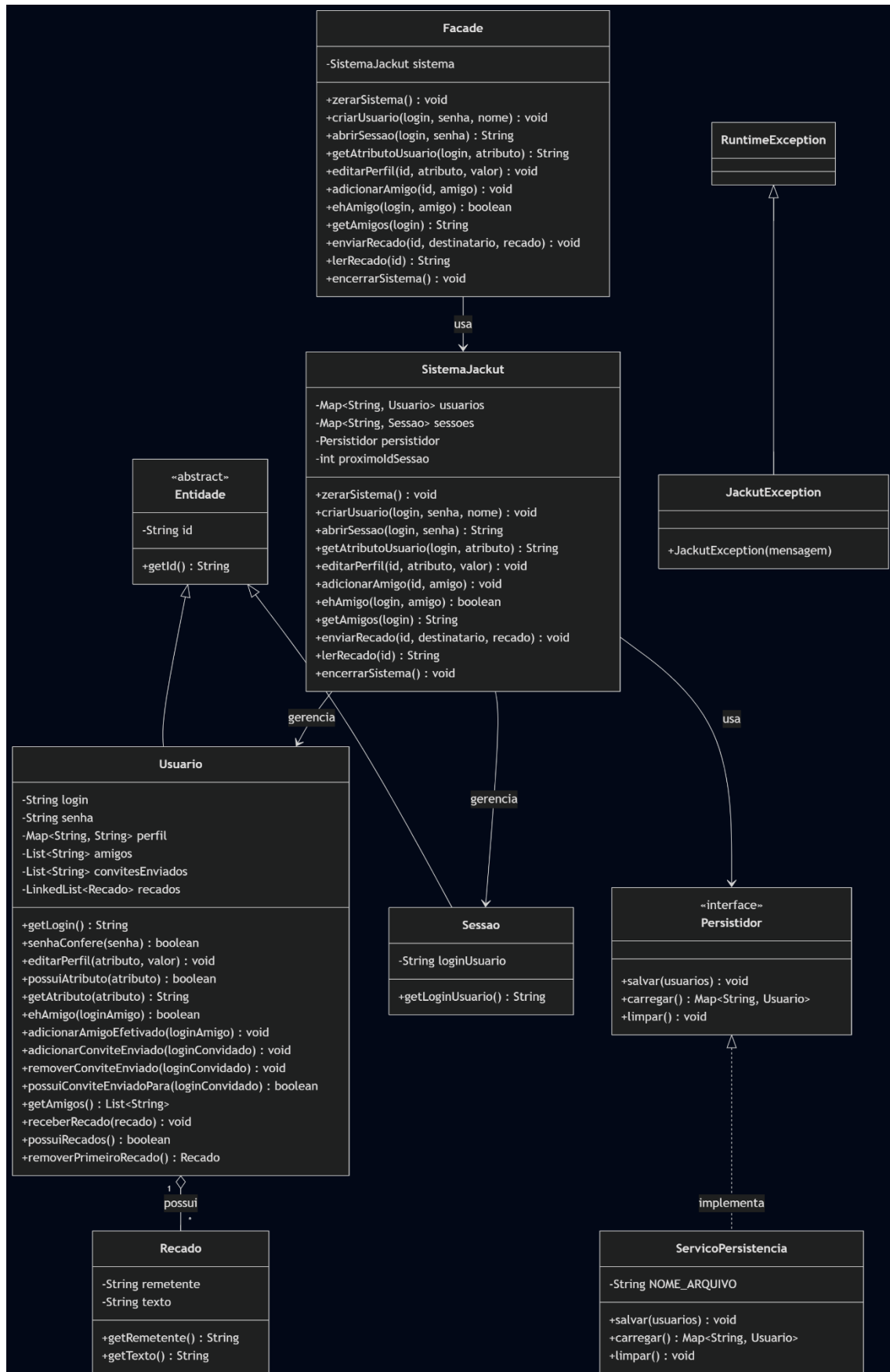
Essa organização permite uma separação clara entre:

- os dados principais do sistema;
- as regras de negócio;
- o tratamento de erros;
- a persistência em arquivo;
- os contratos de comportamento;
- e o ponto de entrada externo do sistema.

No Jackut, os testes de aceitação interagem diretamente com a classe Facade. A Facade, por sua vez, delega as operações para a classe SistemaJackut, que concentra a lógica de negócio referente ao Milestone 01. As classes do pacote modelos representam as entidades fundamentais do domínio, enquanto o pacote servicos contém a lógica central do sistema e a classe responsável pela persistência.

Como o Milestone 01 possui um escopo mais reduzido, optou-se por manter as regras principais na classe SistemaJackut, evitando a criação de um número excessivo de serviços logo no início do projeto. Essa decisão mantém a arquitetura simples e coerente com o tamanho atual do sistema, mas ainda permite expansão futura para novos serviços caso os próximos milestones adicionem comunidades, novos relacionamentos ou remoção de conta.

3. Diagrama de classes



O diagrama de classes acima representa a estrutura principal do sistema Jackut no Milestone 01.

O diagrama apresenta as principais relações entre as classes do projeto:

- Facade utiliza SistemaJackut;
- SistemaJackut gerencia usuários e sessões;
- SistemaJackut depende da interface Persistidor;
- ServicoPersistencia implementa Persistidor;
- Usuario e Sessao herdam da classe abstrata Entidade;
- Usuario possui uma coleção de Recado;
- JackutException representa os erros de regra de negócio do sistema.

4. Principais componentes e suas interações

4.1 Classe Entidade

A classe Entidade representa uma abstração comum para objetos identificáveis do sistema. Ela foi implementada como classe abstrata, pois não existe, no domínio do Jackut, uma entidade genérica que deva ser instanciada diretamente.

Seu principal atributo é:

- id

Essa classe foi criada para aplicar o conceito de abstração e reaproveitar o comportamento comum de entidades que possuem identificador. No Milestone 01, as classes Usuario e Sessao herdam de Entidade.

A criação dessa classe contribui para uma modelagem mais organizada, pois representa uma generalização simples e coerente dentro do domínio.

4.2 Classe Usuario

A classe Usuario representa o usuário cadastrado no Jackut. Ela é uma das principais entidades do sistema, pois concentra os dados necessários para autenticação, perfil, amigos e recados.

Seus principais atributos são:

- login;
- senha;
- perfil;
- amigos;
- convitesEnviados;
- recados.

O atributo perfil foi implementado como um mapa de chaves e valores, permitindo que o usuário cadastre diferentes atributos de perfil, como nome,

descrição, estado civil, aniversário, cidade natal e outros campos exigidos pelos testes. Essa abordagem permite flexibilidade, pois novos atributos podem ser adicionados sem necessidade de alterar a estrutura da classe.

A lista amigos armazena os usuários cuja amizade já foi efetivada. A lista convitesEnviados armazena os convites de amizade ainda pendentes. A fila de recados armazena os recados recebidos pelo usuário, respeitando a ordem de chegada.

A classe aplica encapsulamento ao manter seus atributos privados e oferecer métodos específicos para alterar seu estado. Além disso, o método que retorna amigos devolve uma cópia da lista, evitando que outras classes modifiquem diretamente a estrutura interna do usuário.

4.3 Classe Sessao

A classe Sessao representa uma sessão aberta por um usuário autenticado. Ela armazena o identificador da sessão e o login do usuário associado a ela.

Seus principais atributos são:

- id;
- loginUsuario.

A existência dessa classe permite que operações privadas, como edição de perfil, adição de amigos e envio de recados, sejam realizadas apenas por usuários que tenham uma sessão válida aberta.

Essa modelagem evita que o sistema dependa diretamente do login em todas as operações internas e aproxima o funcionamento do sistema de um fluxo real de autenticação.

4.4 Classe Recado

A classe Recado representa uma mensagem enviada de um usuário para outro dentro do Jackut. Embora os testes de aceitação leiam apenas o texto do recado, a classe também armazena o remetente.

Seus principais atributos são:

- remetente;
- texto.

A criação de uma classe própria para recados evita que o sistema trate mensagens apenas como strings soltas. Isso melhora o design, pois permite representar o conceito de recado como uma entidade do domínio.

Além disso, essa escolha facilita possíveis expansões futuras, como exibir remetente, data de envio ou histórico de mensagens.

4.5 Interface Persistidor

A interface Persistidor define o contrato de persistência do sistema. Ela estabelece as operações que uma classe de persistência deve implementar.

Seus principais métodos são:

- salvar;
- carregar;
- limpar.

O uso dessa interface reduz o acoplamento entre a regra de negócio e a implementação concreta de persistência. A classe SistemaJackut depende do contrato Persistidor, e não diretamente dos detalhes internos de gravação e leitura de arquivo.

Essa decisão permite aplicar o conceito de interface estudado em sala e torna o design mais flexível.

4.6 Classe ServicoPersistencia

A classe ServicoPersistencia implementa a interface Persistidor e é responsável por salvar, carregar e apagar os dados persistidos do sistema.

A persistência foi feita em arquivo local, utilizando o arquivo relativo:

- dados-jackut.ser

Essa classe isola a lógica de entrada e saída de arquivo, evitando que operações de persistência fiquem espalhadas pelo restante do código.

O uso de caminho relativo foi uma decisão importante, pois o projeto será baixado e testado em outra máquina. Assim, o funcionamento do sistema não depende de caminhos absolutos existentes apenas no computador de desenvolvimento.

4.7 Classe SistemaJackut

A classe SistemaJackut é a classe central da lógica de negócio do Milestone 01. Ela controla os usuários cadastrados, as sessões abertas, as regras de perfil, as regras de amizade, os convites pendentes, os recados e a persistência.

Seus principais atributos são:

- usuarios;
- sessoes;
- persistidor;
- proximoldSessao.

A classe é responsável por operações como:

- criar usuário;
- abrir sessão;
- consultar atributo de usuário;
- editar perfil;
- adicionar amigo;
- verificar amizade;
- listar amigos;
- enviar recado;
- ler recado;
- zerar o sistema;
- encerrar o sistema salvando os dados.

Embora concentre as principais regras de negócio do Milestone 01, essa decisão foi tomada considerando o escopo atual do projeto. Como as User Stories implementadas são concentradas em usuários, amizades e recados, a classe `SistemaJackut` consegue manter a lógica organizada sem criar complexidade desnecessária.

Em milestones futuros, caso o sistema seja expandido com comunidades, novos tipos de relacionamento e remoção de conta, a arquitetura poderá ser ampliada com novos serviços especializados.

4.8 Classe `JackutException`

A classe `JackutException` é uma exceção personalizada criada para representar erros de regra de negócio do sistema.

Ela é utilizada em situações como:

- usuário não cadastrado;
- login inválido;
- senha inválida;
- login ou senha inválidos;
- atributo não preenchido;
- tentativa de adicionar a si mesmo como amigo;
- tentativa de adicionar amizade duplicada;
- tentativa de enviar recado para si mesmo;
- tentativa de ler recado inexistente.

Seu uso melhora a clareza do código, pois permite representar erros próprios do domínio do Jackut de maneira mais organizada, sem lançar exceções genéricas diretamente nas regras de negócio.

Além disso, a classe foi adaptada para lidar com diferenças de codificação entre ambientes de execução. Como os testes originais utilizam codificação ISO-8859-1 e algumas máquinas executam o projeto em UTF-8, a mensagem da exceção é ajustada conforme a codificação padrão da JVM, preservando a compatibilidade com os testes do EasyAccept em diferentes computadores.

4.9 Classe `Facade`

A classe Facade é o ponto de entrada principal do sistema. Ela é utilizada pelos testes de aceitação e funciona como uma interface simplificada para o restante da aplicação.

Seu papel não é concentrar toda a lógica, mas sim:

- inicializar o sistema;
- expor os métodos públicos esperados pelos testes;
- delegar chamadas para a classe SistemaJackut;
- controlar operações gerais, como zerar e encerrar o sistema.

A Facade preserva as assinaturas exigidas pelo EasyAccept, funcionando como uma camada de comunicação entre os testes e a lógica interna do projeto.

4.10 Classe Main

A classe Main é responsável por executar os arquivos de teste utilizando o EasyAccept. Ela não contém regra de negócio do sistema, servindo apenas como ponto de execução para validação das funcionalidades implementadas.

No projeto, a classe Main executa os arquivos de teste do Milestone 01 em ordem, respeitando os testes principais e os testes de persistência.

Além disso, a classe Main localiza automaticamente a pasta tests, evitando problemas quando o projeto é baixado ou executado a partir de diretórios diferentes.

5. Interações entre os componentes

O funcionamento do sistema segue um fluxo bem definido.

Os testes de aceitação executam métodos da classe Facade. A Facade, por sua vez, encaminha cada operação para SistemaJackut. A classe SistemaJackut manipula os objetos do domínio, como Usuario, Sessao e Recado. Quando necessário, o sistema utiliza a interface Persistidor, implementada por ServicoPersistencia, para salvar, carregar ou apagar os dados em arquivo.

De forma resumida, o fluxo principal pode ser descrito assim:

Testes → Facade → SistemaJackut → Entidades / Persistência

Essa organização reduz o acoplamento entre os testes e as regras internas do sistema, além de facilitar a manutenção e o entendimento do projeto.

Por exemplo, quando um teste chama o método criarUsuario, a Facade recebe a chamada e delega a operação para SistemaJackut. O sistema valida os dados, cria um objeto Usuario e o armazena no mapa de usuários.

Quando um teste chama `adicionarAmigo`, a `Facade` delega a chamada para `SistemaJackut`, que verifica a sessão, valida se o usuário existe, confere se já existe amizade ou convite pendente e decide se deve criar um convite ou efetivar a amizade.

Quando um teste chama `enviarRecado`, o sistema valida a sessão, verifica o destinatário e adiciona um objeto `Recado` à fila de recados do usuário destinatário.

Quando o teste chama `encerrarSistema`, o sistema utiliza `ServicoPersistencia` para gravar os dados em arquivo, garantindo que os testes de persistência consigam recuperar usuários, perfis, amizades e recados em uma execução posterior.

6. Conceitos de Programação Orientada a Objetos aplicados

6.1 Encapsulamento

Os atributos das classes foram definidos como privados, sendo acessados e modificados por métodos apropriados. Isso impede acesso indevido ao estado interno dos objetos e contribui para manter a consistência dos dados.

A classe `Usuario`, por exemplo, não expõe diretamente suas listas internas. O método `getAmigos` retorna uma cópia da lista, evitando que outras classes modifiquem diretamente a estrutura de amizades do usuário sem passar pelas validações do sistema.

6.2 Herança

Foi aplicada herança entre `Entidade`, `Usuario` e `Sessao`.

A classe `Entidade` representa uma abstração de objetos identificáveis. As classes `Usuario` e `Sessao` herdaram dela porque ambas possuem identificador.

Essa estrutura permite reaproveitar o atributo comum `id` e aplicar herança de forma simples, coerente e alinhada ao domínio do projeto.

6.3 Classe abstrata

A classe `Entidade` foi implementada como abstrata, pois representa uma ideia geral do sistema, mas não um objeto concreto que deva ser instanciado diretamente.

Essa decisão reforça o conceito de abstração estudado em sala, permitindo que características comuns sejam representadas em uma superclasse sem criar uma entidade genérica desnecessária.

6.4 Polimorfismo

O polimorfismo foi aplicado de forma pontual por meio da interface Persistidor.

A classe SistemaJackut trabalha com uma referência do tipo Persistidor, enquanto a implementação concreta utilizada é ServicoPersistencia. Isso permite que o sistema dependa de uma abstração e não de uma classe concreta específica.

Dessa forma, se futuramente fosse necessário trocar a forma de persistência, por exemplo, para banco de dados ou outro formato de arquivo, seria possível criar uma nova implementação de Persistidor sem alterar diretamente a lógica principal do sistema.

6.5 Composição

A composição aparece principalmente na classe Usuario, que mantém coleções relacionadas a outros elementos do sistema, como amigos, convites enviados e recados.

A relação entre Usuario e Recado representa uma relação do tipo “tem vários”, pois um usuário pode possuir vários recados recebidos.

Além disso, SistemaJackut mantém coleções de usuários e sessões, funcionando como a classe controladora do estado principal da aplicação durante a execução.

6.6 Uso de listas

Foram utilizadas listas para armazenar amigos, convites enviados e recados recebidos.

A lista de amigos representa os relacionamentos já efetivados. A lista de convites enviados representa solicitações ainda pendentes. A fila de recados, implementada com LinkedList, permite que os recados sejam lidos na ordem em que foram recebidos.

Isso está alinhado com o conteúdo estudado sobre coleções e listas em Java.

6.7 Tratamento de exceções

O sistema utiliza exceções para sinalizar erros de validação e de regras de negócio. Isso evita retornos inconsistentes e torna a lógica de erro mais organizada e explícita.

A classe JackutException centraliza os erros de domínio e permite que as mensagens exigidas pelos testes sejam lançadas de forma padronizada.

6.8 Interface

A interface Persistidor foi criada para definir o contrato de persistência do sistema. Essa escolha permite que a classe SistemaJackut dependa de uma

abstração, deixando a implementação concreta da persistência isolada em `ServicoPersistencia`.

O uso da interface contribui para reduzir acoplamento e melhorar a organização do design.

7. Padrão de projeto adotado

7.1 Nome do padrão: Facade

O padrão de projeto adotado foi o padrão Facade.

7.2 Descrição geral

O padrão Facade fornece uma interface simplificada para um conjunto mais complexo de classes e subsistemas. Em vez de o código externo interagir diretamente com diversas classes internas, ele passa a interagir com uma única classe centralizadora.

7.3 Problema resolvido

No contexto do projeto, os testes de aceitação precisam acessar o sistema por meio de uma única classe, sem conhecer sua estrutura interna. Se os testes precisassem interagir diretamente com usuários, sessões, recados, persistência e regras internas, o sistema se tornaria mais acoplado e mais difícil de usar externamente.

A existência da Facade resolve esse problema ao expor apenas os métodos esperados pelo `EasyAccept`, mantendo os detalhes internos protegidos nas demais classes.

7.4 Identificação da oportunidade

A oportunidade de aplicar esse padrão surgiu pela própria exigência do projeto, que demanda uma classe Facade compatível com os testes de aceitação.

Assim, havia a necessidade de uma interface única e simplificada para expor as operações do sistema, como criação de usuário, abertura de sessão, edição de perfil, adição de amigos, envio de recados, leitura de recados, zerar sistema e encerrar sistema.

7.5 Aplicação no projeto

A classe Facade foi implementada como ponto de acesso central. Ela expõe os métodos públicos esperados pelos testes, como `criarUsuario`, `abrirSessao`, `editarPerfil`, `adicionarAmigo`, `enviarRecado` e `lerRecado`.

Internamente, ela delega a responsabilidade para SistemaJackut, mantendo a interface externa simples e o design interno mais organizado.

Essa decisão permite que os testes continuem acessando o sistema por uma interface estável, mesmo que a organização interna das classes evolua em milestones futuros.

8. Persistência de dados

A persistência do sistema foi implementada em arquivo local, garantindo que os dados permaneçam salvos entre diferentes execuções do programa.

Essa funcionalidade foi necessária para atender aos testes de persistência do Milestone 01, que verificam se usuários, perfis, amigos e recados continuam disponíveis após o encerramento e reinício do sistema.

A classe `ServicoPersistencia` foi responsável por:

- carregar os dados previamente salvos;
- salvar o estado atual do sistema;
- apagar os dados quando o sistema é reiniciado.

A persistência salva o mapa de usuários. Como cada usuário armazena seus atributos de perfil, sua lista de amigos, seus convites enviados e sua fila de recados, salvar os usuários permite preservar os dados necessários para os testes de persistência.

As sessões não foram persistidas, pois representam um estado temporário da execução. Nos testes de persistência, novas sessões são abertas após o carregamento dos dados salvos.

Essa abordagem foi suficiente para o escopo do projeto e compatível com os testes de aceitação fornecidos.

9. Compatibilidade de execução

Durante a validação do projeto, foi identificado que os arquivos de teste do EasyAccept utilizam codificação ISO-8859-1, enquanto algumas máquinas e configurações do IntelliJ IDEA executam o projeto em UTF-8.

Para evitar falhas de comparação de mensagens acentuadas em ambientes diferentes, a classe `JackutException` adapta as mensagens de erro conforme a codificação padrão da JVM. Essa estratégia permite que o sistema continue compatível tanto em ambientes configurados com ISO-8859-1 quanto em ambientes configurados com UTF-8.

Além disso, a classe `Main` localiza automaticamente a pasta `tests`, evitando dependência de caminhos absolutos ou de uma pasta de execução específica.

Essa decisão atende à recomendação do projeto de utilizar apenas nomes de arquivos relativos e facilita a execução em máquinas diferentes.

10. Testes executados

Foram executados com sucesso os seguintes testes de aceitação do Milestone 01:

- us1_1
- us1_2
- us2_1
- us2_2
- us3_1
- us3_2
- us4_1
- us4_2

Todos os testes foram concluídos com sucesso, comprovando que o sistema implementado atende aos requisitos esperados para esta etapa do projeto.

O total validado foi de 184 testes OK, distribuídos da seguinte forma:

- us1_1: 17 testes OK;
- us1_2: 7 testes OK;
- us2_1: 36 testes OK;
- us2_2: 13 testes OK;
- us3_1: 46 testes OK;
- us3_2: 10 testes OK;
- us4_1: 42 testes OK;
- us4_2: 13 testes OK.

Esses testes validaram criação de contas, autenticação por sessão, edição de perfil, persistência, convites de amizade, efetivação de amizade, envio de recados e leitura de recados.

11. Conclusão

O projeto Jackut – Milestone 01 foi desenvolvido com sucesso em Java, atendendo ao escopo previsto para a disciplina de Programação 2. A solução implementada foi compatível com os testes de aceitação das User Stories 1 a 4 e permitiu aplicar, na prática, conceitos importantes de programação orientada a objetos, como encapsulamento, herança, abstração, composição, uso de listas, exceções, interfaces e organização modular do código.

Além do atendimento funcional, o sistema foi estruturado de forma clara, com separação de responsabilidades entre entidades, serviços, persistência, exceções e fachada principal, o que contribuiu para a legibilidade e manutenção do projeto.

A arquitetura adotada foi mantida simples e adequada ao escopo do primeiro milestone. Ao mesmo tempo, ela oferece uma base coerente para expansão futura, permitindo que novas funcionalidades, como comunidades, mensagens para comunidades, novos tipos de relacionamento e remoção de conta, sejam incorporadas em etapas posteriores do projeto.